1 Tree Evaluation

Consider the function:

$$f : \{0,1\}^k \times \{0,1\}^k \to \{0,1\}^k$$

We aim to find the amount of space needed, given $x_1, ..., x_n \in \{0,1\}^k$ for $n = 2^k$, the root value of the tree defined where each inner node is calculated as:

$$u.val = f(u.left.val, u.right.val)$$

Consider a tree that has 2^k leaf nodes ranging from $x_1, ..., x_{2^k}$, the trivial algorithm to evaluate the root node's value would take $O(k \cdot k)$ space (the height of the tree). The trivial algorithm is as follows:

```
Evaluate(node)
    If node == leaf:
        return node.value
    Else:
        return f(Evaluate(node.left), Evaluate(node.right))
```

The space comes from needing to store the value of $f(u)$ (k bits) for each level which we refer to as h for height where $(h = k)$.

To help understand the problem, we imagine that ever node stores the height at which it is in the tree, and for each level of the tree, we have a register R_i where $1 \leq i \leq k$. Each output will be stored in R_d, where $d = height(u)$. Now we have:

```
    If u is a leaf:
        R1 = u.val
    TEVAL(u.left)
    Rd <- Rd-1
    TEVAL(u.right)
    Rd <- f(Rd-1,Rd)
```

This eliminates the need for recursion. The space here is $\Theta(h \cdot k)$. We assume that $h = k = \log n$

2 Cook-Mertz

A recent result has shown that TEVAL can be solved in $O((k + h) \log k)$ space, or $O(\log n \log \log n)$. This has been applied to show a theorem in

2025 by Ryan Williams that:

$$TIME(t(n)) \subseteq SPACE(\sqrt{t(n)} \cdot polylog(t(n)))$$

2.1 Formula Evaluation With Minimal Extra Registers

Imagine we have a formula as follows:

$$F = (2 + 3) * (6 + 7)$$

To solve this recursively, we can evaluate the formual as a tree. Above, we just reduce the problem to a tree, with leaves being numerica values and the inner nodes representing the operations. We do the same thing where each output is stored in R_d where $d =$ height of u:

```
FEVAL(u):
    If u is leaf:
        R1 = u.val
        return
    FEVAL(u.left)
    Rd <- Rd-1
    FEVAL(u.right)
    If u.op == +:
        Rd <- Rd + Rd-1
    Else:
        Rd <- Rd * Rd-1
```

The space complexity here will be the depth of the tree, h registers for each level of the tree.

In 1989, Barrington showed that regardless of the height of the tree, this formula can be evaluated with a constant number of registers. Ben-Or and Cleave demonstrated it can be done in 3 registers, which is essentially optimal for the root nodes left, right, and own value. It is done with something called a **Straight Line Program** that is executed sequentially without any loops or conditional statements. It also uses **Catalytic Computation**, where memory in recursive calls is somewhat "restored" after the recursive calls are completed. It uses the following definition for a new operation called offset.

$$offset(u, R_i, R_j, R_k, "sign")$$

Where sign is $1/-1$. The Goal for each Offset call is to return the following without changing R_i or R_k:

$$R_j \leftarrow R_j + (sign)R_i \cdot FEVAL_u(x)$$

We have the following for the case where $sign = 1$.

If u == leaf: $R_j \leftarrow R_j + (sign)R_i(u.value)$

If u.op == +: $R_j \leftarrow R_j + R_i(FEVAL_{u.left}(x) + FEVAL_{u.right}(x))$ We compute this by making the following calls:

- $offset(u.left, R_i, R_j, R_k, Sign)$ Which causes the registers value to update to $R_j \leftarrow R_j + R_i \cdot (FEVAL_{u.left}(x))$
- $offset(u.right, R_i, R_j, R_k, Sign)$ Further updates it to $R_j \leftarrow R_j + R_i \cdot (FEVAL_{u.left}(x)) + R_i \cdot (FEVAL_{u.right}(x))$

For u.op == $*$, we want the offset call for u to update registers so that

$$R_j \leftarrow R_j + R_i \cdot FEVAL_{u.left}(x) * FEVAL_{u.right}(x)$$

First, we will rename $FEVAL_{u.left}(x)$ as f and $FEVAL_{u.right}(x)$ as g for simplicity. Our goal is to go from:

$$(R_i, R_j, R_k) \rightsquigarrow (R_i, R_j + R_i \cdot f \cdot g, R_k)$$

We consider the following operations that will help achieve this:

Operation	Updated registers
$R_j \leftarrow R_j - R_k \cdot g$	$(R_i,\ R_j - R_k \cdot g,\ R_k)$
$R_k \leftarrow R_k + R_i \cdot f$	$(R_i, R_j - R_k \cdot g, R_k + R_i \cdot f)$
$R_j \leftarrow R_j + R_k \cdot g$	$(R_i, (R_j - R_k \cdot g) + (R_k + R_i \cdot f) \cdot g, R_k + R_i \cdot f)$ $= (R_i, R_j + R_i \cdot f \cdot g, R_k + R_i \cdot f)$
$R_k \leftarrow R_k - R_i \cdot f$	$(R_i, R_j + R_i \cdot f \cdot g, R_k)$